

Forward Propagation in a Four-Layer Multilayer Perceptron: Design, Implementation, and Verification

Nabonita Das

Course assignment HW#4 — Multilayer Perceptron Implementation

October 2026

ABSTRACT

Abstract—This paper documents the design, implementation and verification of a fully connected multilayer perceptron (MLP) with a 3–5–5–2–2 architecture, in which three sigmoid hidden layers feed a two-neuron ReLU output layer. The network is written from scratch in Python with NumPy and performs forward propagation only; no training is involved. The implementation uses a numerically stable sigmoid, accepts single samples and batches, validates its inputs, and can expose the pre-activation and activation of every layer. Correctness is checked in four independent ways: a hand calculation of one neuron, a second implementation that uses plain loops instead of matrix products, 99 automated tests, and 500 randomized trials in which both implementations agree to within 3.3×10^{-16} . With the default small Xavier-initialised weights, the outputs for 5000 random inputs stay inside a narrow band from 0.203 to 0.312. We show that this band is bounded by the structure of the network, since the last hidden layer lies in the open unit square, and that larger weights widen it to 0.48–5.39 without any change to the code.

Index Terms—multilayer perceptron, feedforward neural network, forward propagation, sigmoid, ReLU, numerical stability, software verification.

I. INTRODUCTION

Multilayer perceptrons are the simplest neural networks that combine several layers of weighted sums with non-linear activation functions, and they remain the building block inside many larger architectures. Before any learning rule can be applied to such a network, its forward computation must be correct: every prediction, loss value and gradient is derived from the signals that the forward pass produces. This paper therefore concentrates on the forward pass alone and treats it as a piece of software that deserves careful specification and testing.

The network studied here has three input features, three hidden layers of five, five and two neurons with sigmoid activations, and an output layer of two neurons with a rectified linear unit (ReLU). The weights and biases may take any values. We implement the network as a Python class, `MultilayerPerceptron`, and ask three questions. Is the implementation correct? Is it numerically robust for extreme inputs? And what does the network actually compute when its parameters are small, as they are under a standard initialisation?

The contributions of this paper are as follows.

- A compact, configurable NumPy implementation of forward propagation with a numerically stable sigmoid, batch support, input validation and a per-layer trace.

- A layered verification strategy that combines a hand calculation, an independent loop-based implementation, 99 automated tests and 500 randomized trials.
- An analysis of the output range of the specified network, including a simple provable bound and an experiment that contrasts small and larger weights.

Section II reviews the required background. Section III states the mathematical model. Section IV describes the implementation, Section V the verification methods, Section VI the results, and Section VII their interpretation and the limitations of the work.

II. BACKGROUND

A. Perceptrons and Multilayer Networks

The idea of computing with simplified neurons goes back to the threshold units of McCulloch and Pitts [1], and Rosenblatt’s perceptron [2] added a learning procedure for a single layer of such units. Stacking several layers produces a multilayer perceptron, whose weights can be learned by back-propagating error gradients [3]. A feedforward network with one hidden layer and a suitable non-linear activation can approximate any continuous function on a compact set to arbitrary accuracy when enough hidden units are available [4, 5]. Deeper networks, such as the one in this paper, are used in practice because they often represent useful functions with fewer units [9, 10].

B. Activation Functions

The logistic sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (1)$$

maps the real line to the open interval $(0, 1)$ and saturates for large $|z|$. The rectified linear unit

$$\text{ReLU}(z) = \max(0, z) \quad (2)$$

is zero for negative arguments and the identity otherwise [7, 8]. Both functions are plotted in Fig. 1. Output activations are normally chosen to match the task, for example linear, sigmoid or softmax [10]. A ReLU at the output, as required here, restricts every prediction to be non-negative and leaves it unbounded above.

C. Parameter Initialisation

Without a learning procedure the parameter values are free, but a principled default is still useful. Glorot and Bengio proposed drawing the weights of a layer with n_{in} inputs and n_{out} outputs

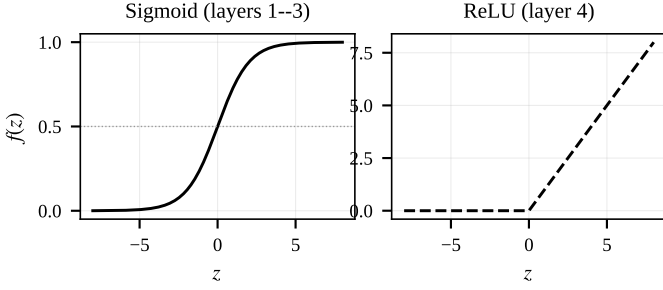


Fig. 1. The two activation functions used by the network. The sigmoid is applied in layers 1–3 and the ReLU in layer 4.

TABLE I
LAYER CONFIGURATION OF THE NETWORK

Layer	Neurons	Activation	W	b	Params
Input	3	–	–	–	0
Hidden 1 (L1)	5	Sigmoid	3×5	5	20
Hidden 2 (L2)	5	Sigmoid	5×5	5	30
Hidden 3 (L3)	2	Sigmoid	5×2	2	12
Output (L4)	2	ReLU	2×2	2	6

uniformly from $[-r, r]$ with $r = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$ [6]. We use this rule for the weights and a small uniform range for the biases.

III. NETWORK ARCHITECTURE AND FORMULATION

Let $n_0, \dots, n_4$ be the layer sizes, here $(3, 5, 5, 2, 2)$, and let $a^{(0)} \in \mathbb{R}^{1 \times 3}$ be the input row vector. Layer l has a weight matrix $W^{(l)} \in \mathbb{R}^{n_{l-1} \times n_l}$, a bias vector $b^{(l)} \in \mathbb{R}^{1 \times n_l}$ and an activation function f_l . The forward pass computes

$$z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)}, \quad (3)$$

$$a^{(l)} = f_l(z^{(l)}), \quad l = 1, \dots, 4, \quad (4)$$

with $f_1 = f_2 = f_3 = \sigma$ and $f_4 = \text{ReLU}$, and the network output is $\hat{y} = a^{(4)} \in \mathbb{R}^{1 \times 2}$. For a batch of N samples the input becomes a matrix $A^{(0)} \in \mathbb{R}^{N \times 3}$ and the bias is added to every row.

Figure 2 shows the connectivity and Fig. 3 the resulting data flow. Table I lists the parameter shapes. The number of trainable parameters is

$$P = \sum_{l=1}^4 (n_{l-1}n_l + n_l) = 20 + 30 + 12 + 6 = 68, \quad (5)$$

and one forward pass for a single sample needs $15 + 25 + 10 + 4 = 54$ multiply–accumulate operations and 14 activation evaluations (12 sigmoids and 2 ReLUs).

IV. IMPLEMENTATION

A. Software Structure

The code is organised as shown in Fig. 4. The code is written in Python with NumPy [11], and all figures were produced with Matplotlib [12]. The module `multilayer_perceptron.py` contains the two activation functions and the class. All other files import from it: a demonstration script, a notebook, two test modules and a smoke test. The constructor takes the layer sizes, the

activation names, an optional random seed, and optional custom weights and biases. The method `forward` accepts one sample or a batch and can also return the pre-activation and activation of every layer. A second method, `forward_pure_python`, repeats the computation with plain loops and is used only for verification.

The core of the forward pass is a loop over the layers:

```
for W, b, f in zip(weights, biases, acts):
    z = a @ W + b
    a = f(z)
```

Because samples are stored as rows, the same code handles a single sample and a batch.

B. Numerically Stable Sigmoid

Evaluating $1/(1 + e^{-z})$ directly overflows the exponential when z is a large negative number. The implementation therefore uses

$$\sigma(z) = \begin{cases} \frac{1}{1 + e^{-z}}, & z \geq 0, \\ \frac{e^z}{1 + e^z}, & z < 0, \end{cases} \quad (6)$$

which is algebraically identical but only ever exponentiates a non-positive number.

C. Initialisation and Input Validation

By default the weights follow the Glorot uniform rule from Section II-C and the biases are drawn from $[-0.1, 0.1]$, using a seeded NumPy random generator so that results are reproducible. Custom weights and biases are accepted if their shapes match the architecture. The class rejects inputs of the wrong rank or feature count and inputs that contain NaN or infinite values, and raises a `ValueError` with a descriptive message in each case.

V. VERIFICATION METHODOLOGY

A. Hand Calculation

As a first check, one neuron is computed by hand. For the input $x = (0.5, -1.0, 2.0)$ the first neuron of layer 1 has weights $(0.47451, 0.82380, -0.22378)$ and bias -0.05455 , so

$$z = 0.5(0.47451) - 1.0(0.82380) + 2.0(-0.22378) + (-0.05455) = -1.08867, \quad (7)$$

and $\sigma(z) = 0.25187$, which equals the value 0.25187 returned by the class.

B. Independent Implementation

The method `forward_pure_python` uses only Python lists, loops and the `math` module. It shares no code with the NumPy path, so agreement between the two is evidence that the vectorised code is correct and not merely self-consistent.

C. Automated Tests

The test suite has three parts. Twenty-one unit tests cover the architecture, parameter shapes and count, known activation values, stability of the sigmoid for extreme arguments, agreement with a manual calculation, and the error cases. Seventy-eight randomized tests use fixed seeds and draw random inputs of mixed

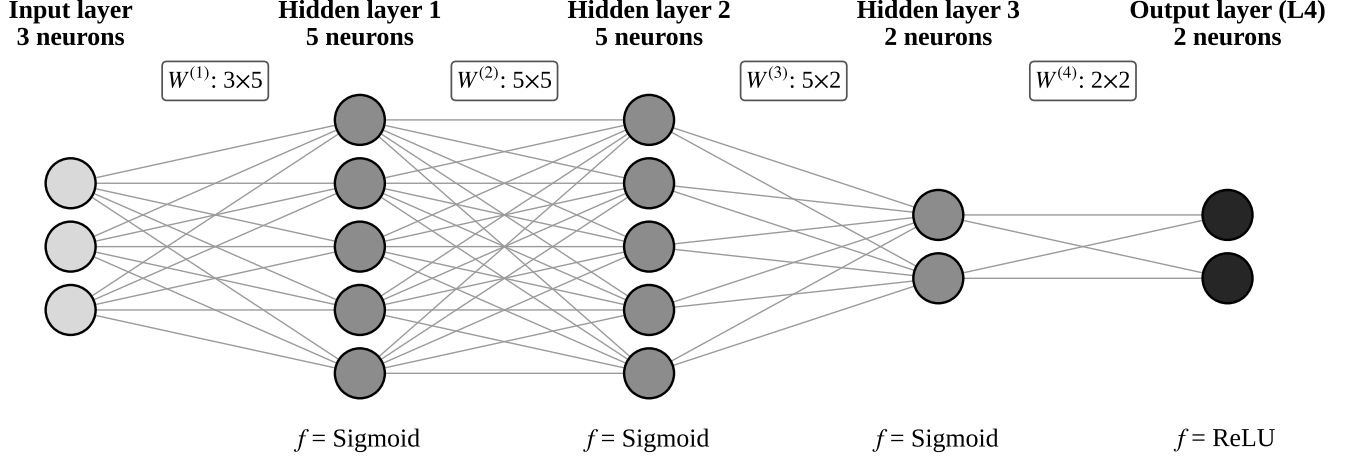


Fig. 2. Architecture of the 3–5–5–2–2 network. Every neuron is connected to every neuron of the next layer. Boxes give the shape of each weight matrix.

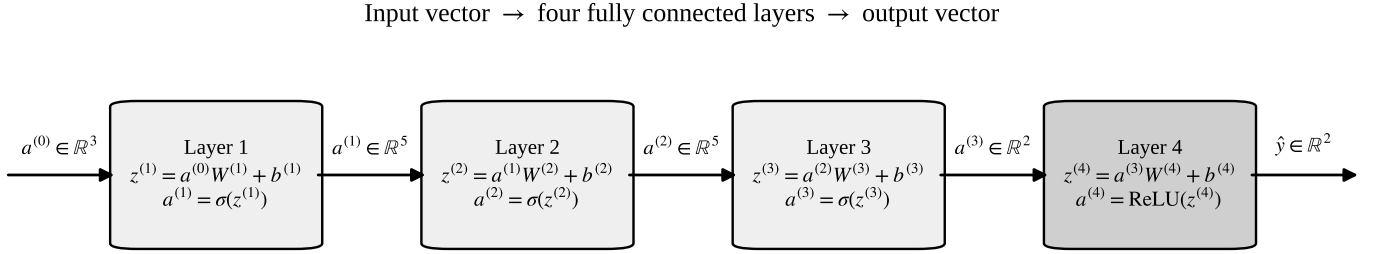


Fig. 3. Data flow of the forward pass. Each block applies an affine map followed by its activation, and the arrows carry the vector dimensions.

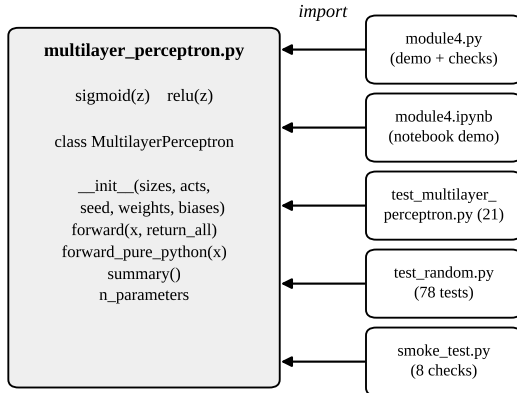


Fig. 4. Software structure. The class lives in one module and is imported by the demonstration script, the notebook and the three test files; numbers give the tests or checks in each.

scale, random batches, and random architectures with random weights; they also check that extreme inputs never overflow, that inputs are not modified, and that repeated calls give identical results. A smoke test runs eight quick end-to-end checks. The demonstration script adds seven checks of its own.

TABLE II
SUMMARY OF VERIFICATION EVIDENCE

Mechanism	Count	Outcome
Hand calculation (one neuron)	1	match, 5 d.p.
Unit tests	21	all pass
Randomized tests (pytest)	78	all pass
Smoke test	8	all pass
Script self-checks	7	all pass
Random trials (NumPy vs. loops)	500	500 pass

D. Randomized Trials

In addition, 500 trials were run in which a new network is created from a random seed and evaluated on an input whose scale is 10^k with k drawn at random from $\{-2, \dots, 2\}$. A trial passes if the output is finite, non-negative and equal to the loop-based result within the default tolerance of `numpy.allclose`.

VI. RESULTS

A. A Worked Example

Table III shows the full forward pass for the input $(0.5, -1.0, 2.0)$ with the default parameters, and Fig. 5 plots the activation of each neuron. The final output is $(0.21184, 0.29902)$. In layer 4 the pre-activations are positive, so the ReLU leaves them unchanged and $z^{(4)} = a^{(4)}$.

TABLE III
LAYER-BY-LAYER TRACE FOR THE INPUT (0.5, −1.0, 2.0)

Layer	Activation	Pre-activation $z^{(l)}$	Activation $a^{(l)}$
1	Sigmoid	−1.089, 0.984, 0.226, 1.999, −0.435	0.252, 0.728, 0.556, 0.881, 0.393
2	Sigmoid	−0.434, −0.016, −0.434, −0.798, 0.651	0.393, 0.496, 0.393, 0.310, 0.657
3	Sigmoid	−0.234, −0.438	0.442, 0.392
4	ReLU	0.212, 0.299	0.212, 0.299

TABLE IV
NETWORK OUTPUT FOR EXAMPLE INPUTS

Case	Input (x_1, x_2, x_3)	Default weights		Scaled weights	
		$\hat{y}_1$	$\hat{y}_2$	$\hat{y}_1$	$\hat{y}_2$
All zeros	(0, 0, 0)	0.21212	0.30011	3.034	4.326
All ones	(1, 1, 1)	0.20686	0.29293	2.463	3.627
Mixed	(0.5, −1, 2)	0.21184	0.29902	1.771	2.878
Negative	(−3, −2, −1)	0.22032	0.31186	3.436	4.782
Large	(50, −50, 100)	0.21094	0.30027	1.221	2.241

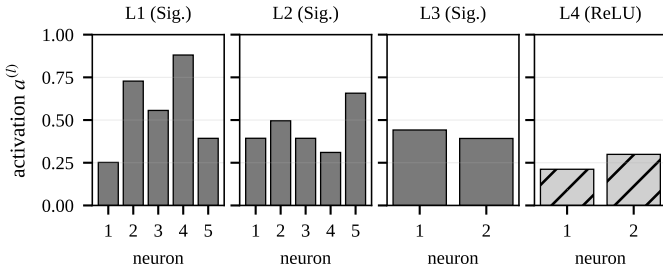


Fig. 5. Activation of every neuron after each layer for the input (0.5, −1.0, 2.0). Sigmoid layers (L1–L3) stay inside (0, 1); the ReLU layer (L4, hatched) gives the network output.

B. Further Example Inputs

Table IV lists the output for five inputs that span zeros, ones, mixed signs, negative values and very large values. The left pair of output columns uses the default parameters and the right pair the larger demonstration weights introduced in Section VI-D.

C. Behaviour with the Default Weights

All outputs in Table IV lie between 0.2 and 0.32 for the default network, even though the inputs differ by several orders of magnitude. Figure 6(a) confirms this: sweeping x_1 from −6 to 6 moves each output by less than 0.03. Over 5000 random standard-normal inputs, the outputs stay between 0.203 and 0.312 (Fig. 8(a)); the two clusters in that histogram are the two outputs, each concentrated near its own value.

D. Scaled Demonstration Weights

To show that this narrow range comes from the parameters and not from the code, a second network is built with the same class. Its weights are drawn from a normal distribution with standard deviation 3.0 (seed 277), and the output-layer weights and biases are made non-negative so that the ReLU neurons stay active. For the same sweep the outputs now range from 0.63 to 5.16 (Fig. 6(b)), and Fig. 7 shows a clearly non-linear surface over two inputs, in which a steep transition separates a region of high

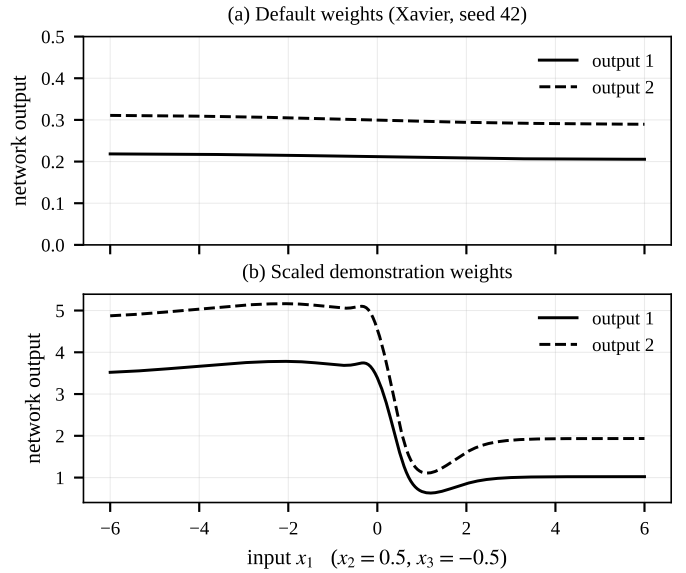


Fig. 6. Network output as the first input is swept from −6 to 6 with the other inputs fixed. (a) Default weights: the response is almost flat. (b) Larger demonstration weights: the response is clearly visible.

output from a region of low output. Over 5000 random inputs the outputs span 0.48 to 5.39 (Fig. 8(b)).

E. Numerical Stability

With floating-point overflow, invalid-operation and divide-by-zero conditions configured to raise exceptions, the input $(10^6, -10^6, 10^6)$ is processed without error and returns (0.21094, 0.30027). These values equal those of the “large” input (50, −50, 100) in Table IV to five decimal places, which is consistent with saturation of the first-layer sigmoids at 0 or 1.

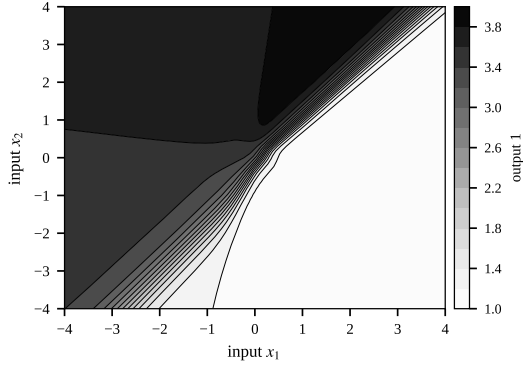


Fig. 7. Output 1 of the demonstration network over a grid of the first two inputs, with the third input fixed at zero.

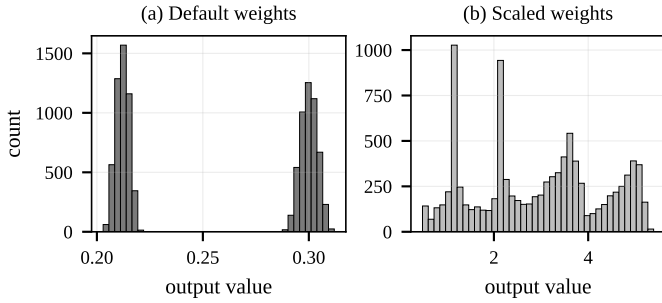


Fig. 8. Distribution of both outputs over 5000 random standard-normal inputs. (a) Default weights. (b) Scaled weights.

VII. DISCUSSION

A. Why the Default Output Range Is Narrow

The narrow range is not accidental. The last hidden layer uses a sigmoid, so $a^{(3)} \in (0, 1)^2$. Each output is therefore bounded by

$$\hat{y}_j \leq \max\left(0, \sum_i \max(W_{ij}^{(4)}, 0) + b_j^{(4)}\right), \quad (8)$$

because every $a_i^{(3)}$ lies between zero and one, so each term $a_i^{(3)}W_{ij}^{(4)}$ is at most $\max(W_{ij}^{(4)}, 0)$. For the default parameters this gives $\hat{y}_1 \leq 0.502$ and $\hat{y}_2 \leq 0.741$, and for the demonstration weights $\hat{y}_1 \leq 4.34$ and $\hat{y}_2 \leq 5.89$. The observed ranges lie inside these bounds. Small weights add a second effect: the pre-activations stay close to the biases, where the sigmoid is nearly linear and varies little. A network of this shape therefore needs larger weights, or an output layer with more freedom, to produce a wide range of outputs.

B. The ReLU Output Layer

Because the output activation is a ReLU, an output can be exactly zero whenever its pre-activation is not positive, and in that case it carries no information about how negative the pre-activation was. This is illustrated by setting all weights to 0.1, every bias to zero except $b_1^{(4)} = -5$, and using the input $(1, 1, 1)$; the output is then $(0.0, 0.11419)$. Such behaviour is acceptable for non-negative targets, but it should be kept in mind if the network is later trained, because a neuron that stays at zero receives no gradient.

C. Limitations and Future Work

The work is limited to the forward pass: the network is not trained, no loss function or back-propagation is implemented, and the demonstration weights were chosen by hand to show a visible response rather than to solve a task. Only dense layers with sigmoid and ReLU activations are supported, and all arithmetic uses 64-bit floating point. Natural extensions are a backward pass verified by numerical gradient checking, further activation functions, and mini-batch training on a small data set.

VIII. CONCLUSION

A four-layer MLP with a 3–5–5–2–2 architecture, sigmoid hidden layers and a ReLU output layer was implemented from scratch and verified by a hand calculation, an independent loop-based implementation, 99 automated tests and 500 randomized trials. The implementation is numerically stable for extreme inputs and agrees with the independent version to within 3.3×10^{-16} , which is at the level of double-precision rounding. The default small weights produce outputs confined to a narrow band that is bounded by the structure of the network, while larger weights produce a clearly visible response from the same code. These results provide a verified starting point for adding training to the network.

REFERENCES

- [1] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *Bull. Math. Biophys.*, vol. 5, no. 4, pp. 115–133, 1943.
- [2] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychol. Rev.*, vol. 65, no. 6, pp. 386–408, 1958.
- [3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [4] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Math. Control Signals Syst.*, vol. 2, no. 4, pp. 303–314, 1989.
- [5] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.
- [6] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. 13th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2010, pp. 249–256.
- [7] V. Nair and G. E. Hinton, “Rectified linear units improve restricted Boltzmann machines,” in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 807–814.
- [8] X. Glorot, A. Bordes, and Y. Bengio, “Deep sparse rectifier neural networks,” in *Proc. 14th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2011, pp. 315–323.
- [9] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [11] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [12] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.